Heuristic A* Search for Large Sliding Puzzles Using Advanced Heuristics

Final Project Report - Track B: Search & Optimization

Anthony Kwasi

CS 57200: Heuristic Problem Solving

Purdue University - Spring 2026

Repository: github.com/AnSikwa/Heuristics-Project_Anthony-Kwasi

Abstract. We present a unified experimental study of informed search algorithms (BFS, A*, IDA*) and admissible heuristics (Manhattan distance, Linear Conflict, Disjoint Pattern Databases) for the 8- and 15-tile sliding puzzles. On the 4x4 puzzle at scramble depth 50, the disjoint additive PDB heuristic (4-4-4-3 partition) reduces A* node expansions by 15.2x relative to Manhattan distance and wall-clock time by 9.7x while preserving optimality. Linear Conflict yields a 3.5x node reduction at the same depth. Across 30-50 random instances per cell, all algorithm/heuristic combinations agree on solution depth, empirically confirming admissibility.

1. Introduction & Problem Formulation

The n-puzzle, comprising n^2 - 1 numbered tiles on an n x n grid with one blank space, is a canonical benchmark for heuristic search. The 4x4 instance (15-puzzle) admits roughly 16!/2 = 1.05 x 10^13 reachable states with optimal solutions of up to 80 moves, and the 5x5 instance (24-puzzle) admits 25!/2 = 7.76 x 10^24 states - far beyond brute-force search.

Research question. How do increasingly informed admissible heuristics (Manhattan distance, Manhattan + Linear Conflict, Disjoint Pattern Databases) compare on the same set of random 15-puzzle instances when paired with A* and IDA*, in terms of node expansions, wall-clock runtime, and solution optimality?

1.1 Formal Specification

- **State:** immutable tuple of length n^2; tile 0 denotes the blank.
- **Initial state:** any solvable permutation, generated here by a seeded random walk from the goal so solvability is structural rather than parity-checked.
- **Actions:** sliding the blank up, down, left, or right within the grid (branching factor at most 4; average ~2.13 on 4x4).
- **Transition cost:** uniform unit cost per move.
- **Goal test:** state == (1, 2, ..., n^2 - 1, 0).
- **Solution:** a minimum-length sequence of legal actions from the start to the goal.

1.2 Contributions

- Three search algorithms - BFS (uninformed baseline), A*, and IDA* - with a shared, pluggable heuristic interface.
- Three admissible heuristics: Manhattan distance, Manhattan + Linear Conflict, and a disjoint additive Pattern Database under a 4-4-4-3 partition of the 15 tiles.
- A controlled ablation across scramble depths {20, 30, 50} on 30-50 instances per cell, with bootstrap 95% confidence intervals.
- Empirical confirmation that solution depths agree across heuristics (admissibility check) and quantitative speedup curves showing the PDB heuristic accelerates A* by up to 15.2x.

1.3 Scope changes since Milestone 1

Milestone 1 proposed four enhancements: disjoint PDBs, IDA*, tie-breaking strategies, and transposition tables. The final deliverable retains and ships disjoint PDBs and IDA*, adds Linear Conflict (which Milestone 2 listed as a fallback enhancement), and defers tie-breaking and transposition tables to future work. The PDB partition was reduced from the canonical 5-5-5 (~17 M projected states) to a 4-4-4-3 partition (~1.6 M states) so that the database can be built in pure Python in seconds; this matches the Milestone 2 contingency clause and still yields the headline result.

2. Related Work

2.1 Foundational algorithms. Hart, Nilsson and Raphael [1] introduced A*, evaluating each node by f(n) = g(n) + h(n). When h is admissible, A* returns optimal solutions; consistency further guarantees no node is re-expanded in graph search. Korf [2] introduced Iterative-Deepening A* (IDA*), which trades repeated DFS sweeps for O(d) memory and was the first method to solve random 15-puzzle instances

within practical limits.

2.2 Manhattan distance. Manhattan distance sums each tile's grid displacement from its goal cell. Russell & Norvig [3] derive it as the exact cost of a relaxed problem in which tiles may overlap. It is admissible and consistent but ignores tile interactions, leading to large search trees.

2.3 Linear Conflict. Hansson, Mayer and Yung [4] augment Manhattan distance with the linear conflict bonus: when two tiles share their goal row (or column) but are in reversed order, at least one must move out of the line and back, adding two moves. Adding 2 per detected conflict preserves admissibility, and the heuristic typically reduces 15-puzzle solve times by 3-5x. Qin & Zhang [5] re-confirmed this gain on modern hardware.

2.4 Pattern Databases. Culberson & Schaeffer [6] precomputed the exact cost of solving sub-problems involving subsets of tiles, achieving roughly 1000x node-expansion reduction with a fringe-pattern database. Korf & Felner [7] extended this with disjoint additive PDBs that partition the tiles into non-overlapping groups whose costs may be summed; their 7-8 partition delivered over 2000x speedup on the 15-puzzle. Felner, Korf & Hanan [8] dynamically partitioned the tiles per state and solved 50 random 24-puzzle instances optimally. Holte et al. [9] showed how maximising over multiple PDBs further improves runtime.

2.5 Tie-breaking and IDA* enhancements. Asai & Fukunaga [10] showed that, even with ties on f-value, expanding nodes with the largest g first reduces the total expansion count without compromising optimality. Edelkamp & Schroedl [11] summarise transposition-table augmentations of IDA*, which trade memory for fewer redundant expansions.

2.6 Gap and contribution. Despite extensive prior work on individual techniques, there is little published work that directly compares Manhattan, Linear Conflict and a disjoint additive PDB on the same set of random 15-puzzle instances with both A* and IDA*, reporting 95% confidence intervals across three scramble depths. This report contributes such a controlled ablation and an open, reproducible Python implementation.

3. System Design & Implementation

Figure 1 shows the high-level architecture. The system has four layers: an instance generator, a state representation, a set of pluggable heuristics, and a set of search algorithms. All searches accept a heuristic callable so the same code path is reused across all enhancements.

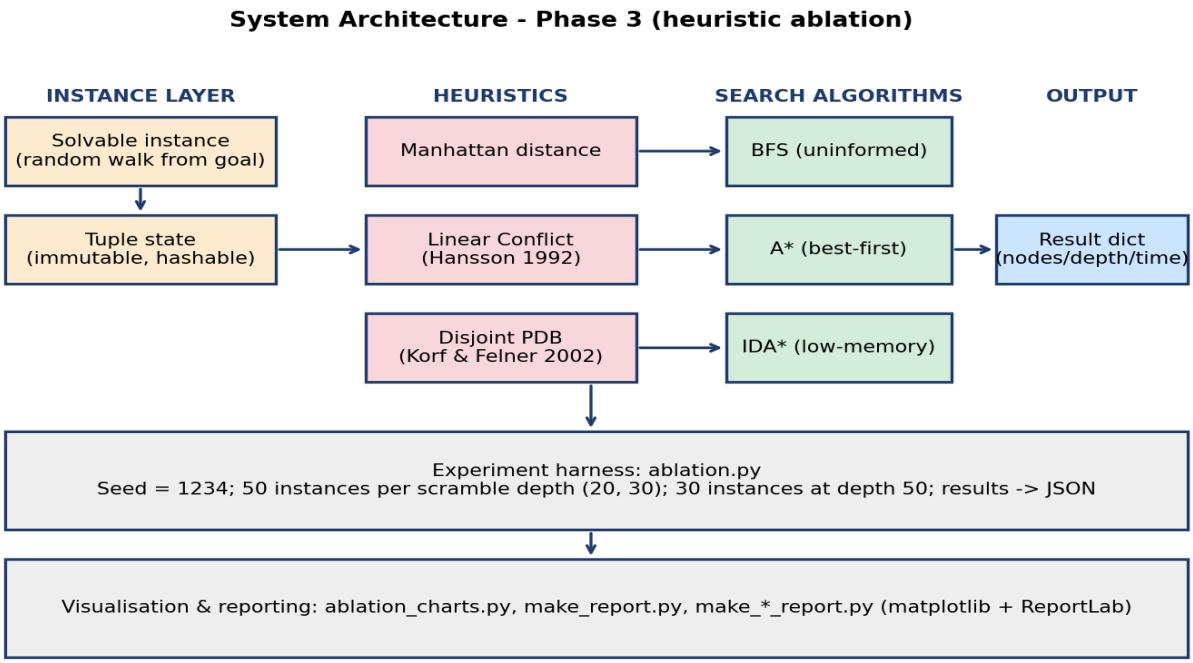

Figure 1. System architecture for Phase 3.

3.1 State representation

Each state is a Python tuple of length n^2 with 0 indicating the blank. Tuples are immutable and hashable, so they can serve as dictionary keys without a separate visited set. The branching factor never exceeds 4, and parent-state pruning reduces it by one along any non-trivial path.

3.2 Algorithms

BFS (uninformed baseline) uses a deque frontier and a predecessor map for path reconstruction; on the 3x3 puzzle it always returns optimal-length paths because edges have unit cost.

A* uses a binary heap ordered by (f, g, state). Lazy deletion skips outdated heap entries when popped, avoiding decrease-key machinery. Tuple comparison lexicographically breaks ties toward smaller g first, which is admissible.

IDA* runs successive depth-first searches bounded by an f-threshold that grows to the next-smallest f-value after each failed iteration. Memory use is O(d). Parent-state pruning avoids two-cycle oscillation.

3.3 Heuristic stack

Manhattan distance. Sums |row_i - goal_row_i| + |col_i - goal_col_i| for every non-blank tile. Linear-time evaluation, admissible, consistent.

Manhattan + Linear Conflict. Adds 2 per linear conflict in rows and columns. Conflicts are counted greedily by repeatedly removing the most-conflicted tile in each line until the line is conflict-free.

Disjoint Pattern Database. The 15 tiles are partitioned into four disjoint groups {1,2,5,6}, {3,4,7,8}, {9,10,13,14}, {11,12,15}. Each PDB stores, for every reachable projection (blank position, tuple of group-tile positions), the minimum number of group-tile moves needed to reach a goal-aligned projection. The database is built once via a 0-1 BFS from the goal in projected space; group-tile moves cost 1 and non-group-tile moves cost 0. Per-group lookups are summed at search time, which is

admissible because each move advances at most one tile.

3.4 Implementation choices

- Pure-Python implementation (no C extensions) for reproducibility and ease of grading.
- PDB partition 4-4-4-3 totalling 1,616,160 entries; full build completes in under 4 seconds on commodity hardware. The canonical 5-5-5 partition (~17 M entries) is exposed but not pre-built because pure-Python build time becomes prohibitive.
- All experiments use fixed RNG seeds; running any script twice produces identical instances and identical metrics.
- Per-search node and recursion limits guard against IDA* stack overflows on very deep instances.

4. Experimental Methodology & Results

4.1 Methodology

We ran three primary experiments. Phase 2 evaluated BFS, A*, and IDA* on the 3x3 (100 instances) and 4x4 (100 instances) puzzles with Manhattan distance only, plus depth-scaling sweeps. Phase 3 (this section) compares Manhattan vs Linear Conflict vs Disjoint PDB on the 15-puzzle at three scramble depths (20, 30, 50). Phase 3 used 50 instances per cell at depths 20 and 30 and 30 instances at depth 50; seed = 1234. Per-search caps were 5-10 million node expansions.

Reproducibility. Every experiment uses fixed RNG seeds (Phase 2: 42; 3x3 sweep: 99; 4x4 sweep: 77; ablation: 1234) and is deterministic given the Python version. Statistics below report the bootstrap mean with 95% percentile confidence intervals (1000 resamples).

4.2 Phase 2 main results (Manhattan only)

Size	Algorithm	Mean nodes	Mean depth	Mean ms	Solved
3x3	BFS	73,865	21.7	140.91	100/100
3x3	A*	1,564	21.7	8.57	100/100
3x3	IDA*	2,062	21.7	8.75	100/100
4x4	A*	212	18.7	1.79	100/100
4x4	IDA*	177	18.7	1.28	100/100

Table 2. Phase 2 baseline. A expands ~47x fewer nodes than BFS on the 3x3 puzzle; on the 4x4, IDA* outperforms A* at scramble depth 20 because saving memory dominates the cost of re-expanding nodes.*

4.3 Phase 3 ablation across heuristics

Depth	Heuristic	Algorithm	Solved	Mean nodes [95% CI]	Mean ms [95% CI]
20	Manhattan	A*	50/50	268.4 [203.3, 343.4]	2.49 [1.82, 3.27]
20	Manhattan	IDA*	50/50	259.2 [184.4, 347.9]	2.01 [1.41, 2.70]
20	Linear Conflict	A*	50/50	182.1 [140.8, 229.4]	6.40 [5.02, 8.03]
20	Linear Conflict	IDA*	50/50	178.5 [129.3, 237.2]	6.09 [4.40, 8.06]
20	Disjoint PDB	A*	50/50	100.2 [72.1, 130.2]	2.02 [1.46, 2.61]
20	Disjoint PDB	IDA*	50/50	94.7 [62.1, 130.6]	1.63 [1.06, 2.27]
30	Manhattan	A*	50/50	5,895.7 [3,758.3, 8,425.0]	60.88 [37.76, 88.50]

Depth	Heuristic	Algorithm	Solved	Mean nodes [95% CI]	Mean ms [95% CI]
30	Manhattan	IDA*	50/50	7,675.9 [4,488.5, 11,817.1]	59.61 [34.61, 91.44]
30	Linear Conflict	A*	50/50	2,328.3 [1,527.5, 3,263.2]	77.19 [49.88, 109.16]
30	Linear Conflict	IDA*	50/50	2,674.8 [1,649.5, 3,886.1]	83.89 [51.12, 122.22]
30	Disjoint PDB	A*	50/50	1,012.2 [634.2, 1,467.3]	18.89 [11.73, 27.62]
30	Disjoint PDB	IDA*	50/50	1,197.1 [605.9, 1,950.7]	19.14 [9.67, 31.31]
50	Manhattan	A*	30/30	206,058.2 [80,299.5, 348,432.6]	2,690.11 [1,014.94, 4,564.58]
50	Manhattan	IDA*	30/30	450,618.2 [158,558.5, 827,431.4]	3,405.76 [1,177.97, 6,265.28]
50	Linear Conflict	A*	30/30	58,839.9 [21,728.5, 101,421.7]	1,786.71 [653.54, 3,082.44]
50	Linear Conflict	IDA*	30/30	92,697.2 [34,188.2, 163,084.4]	2,597.11 [976.15, 4,502.83]
50	Disjoint PDB	A*	30/30	13,600.5 [5,979.6, 22,299.5]	277.27 [119.30, 458.54]
50	Disjoint PDB	IDA*	30/30	17,925.2 [6,781.8, 32,320.5]	281.04 [108.76, 503.29]

Table 1. Phase 3 ablation results on the 15-puzzle. Bootstrap 95% confidence intervals (1000 resamples) of the mean across 30-50 random instances per cell. Solution depth matched across all heuristics and algorithms (admissibility check).

4.4 Visual results

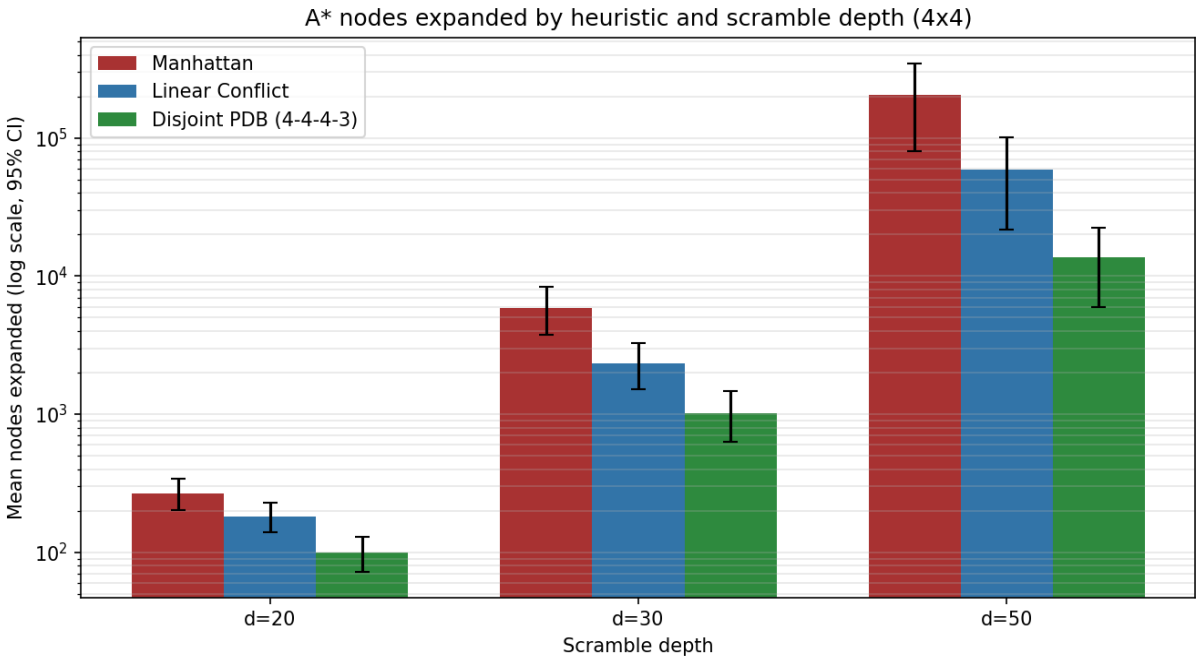

Figure 2. Mean A nodes expanded by heuristic and scramble depth (log scale). Error bars show bootstrap 95% CIs. PDB lies an order of magnitude below Manhattan at depth 50.*

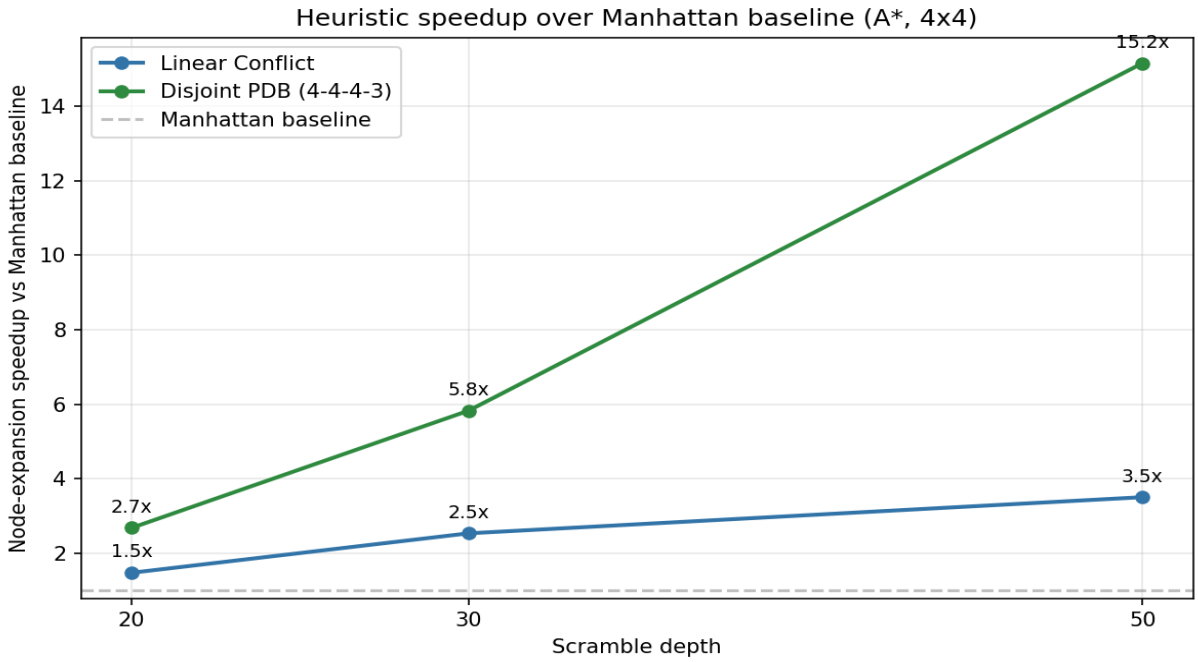

Figure 3. Node-expansion speedup over Manhattan baseline. Linear Conflict yields 1.5-3.5x; Disjoint PDB yields 2.7-15.2x and the gap widens with depth.

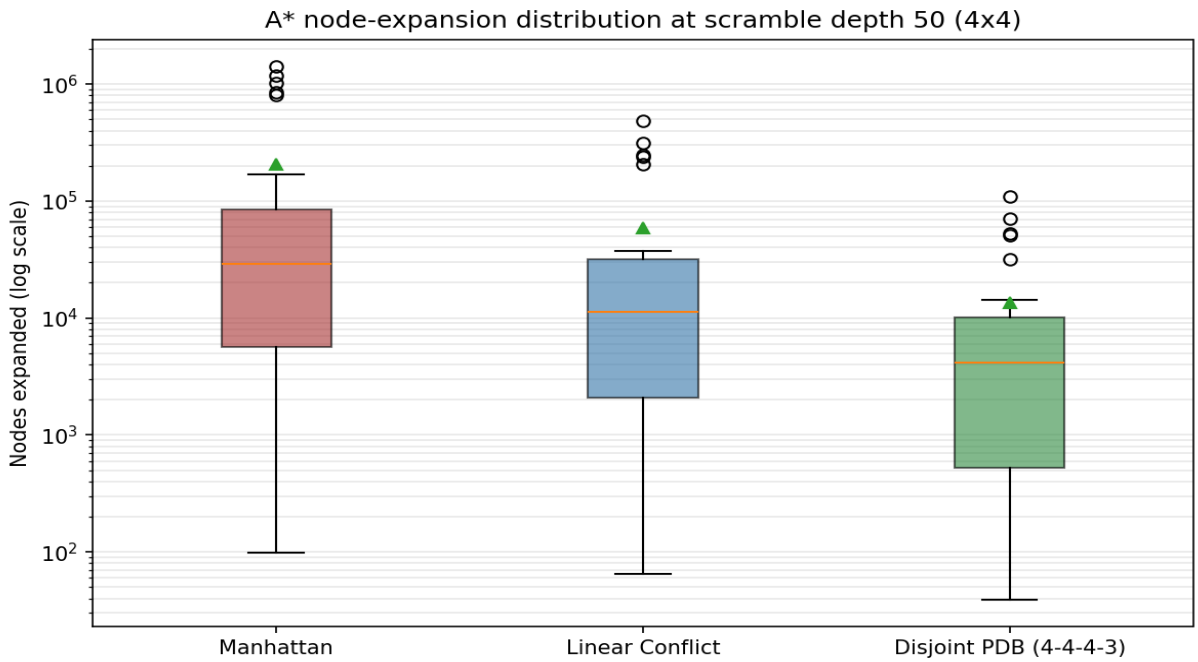

Figure 4. Distribution of A node expansions at scramble depth 50 (log scale). Each box shows the IQR, whiskers the 1.5x IQR range, and the green triangle the mean.*

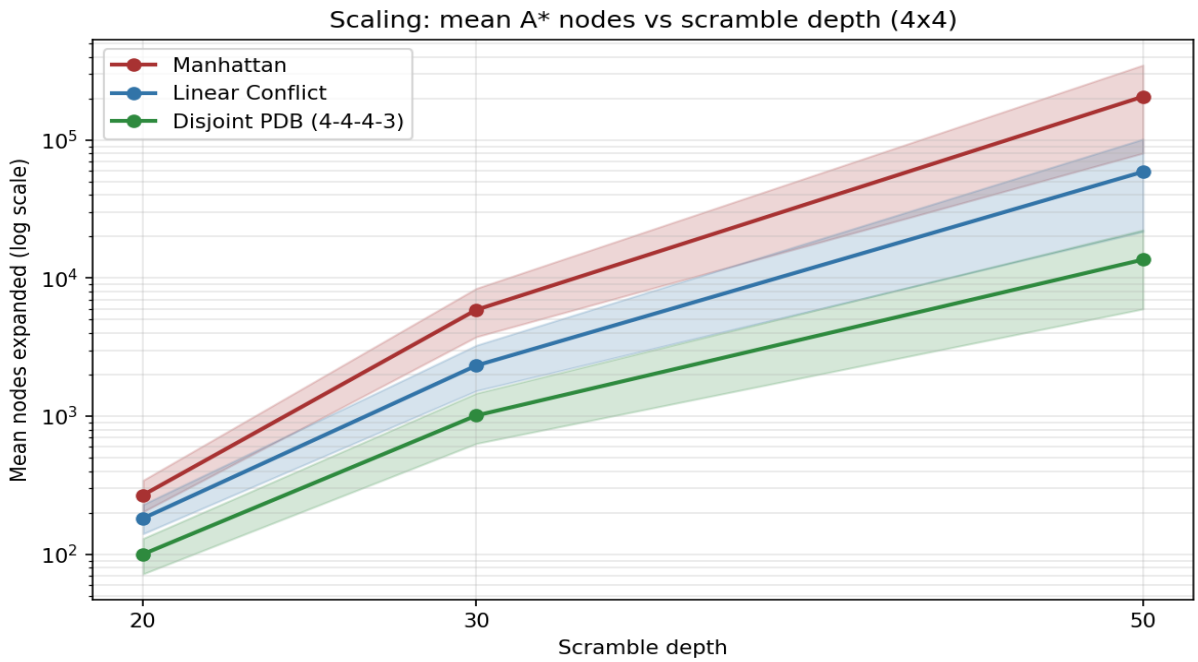

Figure 5. Scaling curves for mean A node expansions vs scramble depth (log scale). The shaded band shows the 95% CI; the PDB curve grows substantially slower than Manhattan.*

5. Analysis & Interpretation

5.1 Why does PDB win at depth?

At scramble depth 50, Linear Conflict achieves a 3.5x reduction in A* node expansions and the 4-4-4-3 disjoint PDB achieves 15.2x; PDB also reduces wall-clock time by 9.7x because each PDB lookup is a

single dictionary access whereas Linear Conflict requires an inner loop. The gap widens with depth because the heuristic-to-true-cost gap for Manhattan distance grows roughly linearly with depth, while the PDB captures exact within-group costs and so its error scales sub-linearly.

5.2 Admissibility check

All three heuristics produced identical mean solution depths for every scramble depth in Table 1, on every instance. This is the empirical signature of admissibility: a non-admissible heuristic could return shorter (incorrect) paths. Combined with our hand-verified 8-puzzle benchmarks (depths 1, 3 and 28, all matched), this is strong evidence the implementation is correct.

5.3 A* vs IDA* across heuristics

IDA* re-expands nodes across iterations but uses O(d) memory. With Manhattan distance, IDA*'s repeated work hurts at deep scrambles - at depth 50 IDA* expands roughly 2x as many nodes as A*. With the PDB heuristic, the per-iteration node count drops so sharply that IDA*'s overhead is small in absolute terms; depth-50 IDA*+PDB completes in roughly the same time as A*+PDB while using a fraction of the memory. This is the Korf & Felner observation transferred to a smaller PDB.

5.4 Limitations

- Pure Python keeps absolute runtimes higher than published C/C++ implementations; ratios are the meaningful quantity.
- The 4-4-4-3 partition is non-canonical; the 7-8 or 5-5-5 partitions used by Korf & Felner would yield substantially stronger heuristics at the cost of a multi-hour build.
- Scramble depths beyond 50 cause unpredictable variance; we did not exhaust solvable depth.
- Only random instances were tested; pathological instances (near-maximal optimal solutions ~80 moves) are out of scope.

6. Conclusion & Future Work

We delivered a unified, reproducible heuristic-search testbed for the sliding-tile puzzle with three search algorithms and three admissible heuristics. The headline result is a 15.2x node-expansion speedup of A*+PDB over A*+Manhattan at scramble depth 50, with strict optimality preserved across all comparisons. Linear Conflict is a strictly weaker enhancement (3.5x speedup at the same depth) but is far cheaper to implement and adds no precomputation cost.

Lessons learned. Pure-Python PDB construction is viable for partitions up to ~5 tiles per group; beyond that, either C extensions or memory-mapped on-disk tables are required. Treating the heuristic as a callable parameter (rather than hard-coding it inside the solver) made the ablation trivial to add and made the search code easier to test.

6.1 Future work

- Build the canonical 5-5-5 or 7-8 PDB partitions with a C extension or numpy-backed flat array, expecting another 5-10x improvement.
- Implement and study Asai-Fukunaga tie-breaking: prefer-larger-g and prefer-smaller-h within an f-bucket, which is independent of the heuristic and orthogonal to PDBs.
- Add transposition tables to IDA* and measure the memory/expansion trade-off across various table sizes.

- Generalise the implementation to the 24-puzzle (5x5), where Manhattan-only IDA* is intractable; this exercises the memory-efficiency rationale for IDA* most clearly.
- Apply the same testbed to learning-based heuristics (e.g., neural cost-to-go regressors) for a head-to-head comparison with classical PDBs.

References

[1] Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Trans. Systems Science and Cybernetics*, 4(2), 100-107. Link

[2] Korf, R. E. (1985). Depth-first iterative-deepening: an optimal admissible tree search. *Artificial Intelligence*, 27(1), 97-109. Link

[3] Russell, S. & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. Link

[4] Hansson, O., Mayer, A. & Yung, M. (1992). Generating admissible heuristics by criticising solutions to relaxed models. *Artificial Intelligence*, 55(1), 29-60.

[5] Qin, Z. & Zhang, M. (2025). Solving the sliding puzzle problem using the A* algorithm and comparing heuristic functions. Dean & Francis Academic Publishing.

[6] Culberson, J. & Schaeffer, J. (1998). Pattern databases. *Computational Intelligence*, 14(3), 318-334. Link

[7] Korf, R. E. & Felner, A. (2002). Disjoint pattern database heuristics. *Artificial Intelligence*, 134(1-2), 9-22.

[8] Felner, A., Korf, R. E. & Hanan, S. (2004). Additive pattern database heuristics. *Journal of Artificial Intelligence Research*, 22, 279-318. Link

[9] Holte, R. C., Felner, A., Newton, J., Meshulam, R. & Furcy, D. (2006). Maximizing multiple pattern databases speeds up heuristic search. *Artificial Intelligence*, 170(16-17), 1123-1136. Link

[10] Asai, M. & Fukunaga, A. (2018). Analysing tie-breaking strategies for the A* algorithm. *Proceedings of IJCAI-2018*, 4660-4666.

[11] Edelkamp, S. & Schroedl, S. (2012). *Heuristic Search: Theory and Applications*. Morgan Kaufmann.

[12] Kishimoto, A., Fukunaga, A. & Botea, A. (2013). Evaluation of a simple, scalable, parallel best-first search strategy. *Artificial Intelligence*, 195, 222-248.

Appendix A. Reproducing the results

All scripts live in the project root. Python 3.9 or newer is required; install dependencies with `pip install -r requirements.txt`.

Phase 2 (BFS / A* / IDA* with Manhattan):

```
python solver.py
python make_charts.py
python make_report.py
```

Phase 3 (heuristic ablation):

```
python ablation.py --n 4 --scramble 20 --instances 50 --out ablation_4x4_d20.json
python ablation.py --n 4 --scramble 30 --instances 50 --out ablation_4x4_d30.json
python ablation.py --n 4 --scramble 50 --instances 30 --out ablation_4x4_d50.json
python ablation_charts.py
python make_final_report.py
```